

Python Full-Stack with Django +

AI/ML *Complete Syllabus & Roadmap*

Student Path: Django → DRF → Fullstack → FastAPI → AI/ML

Total Duration	12-18 months (self-paced)
Target	Python Full-Stack Developer + AI/ML Integration
Frameworks	Django, DRF, React (frontend), FastAPI
End Goal	Build & deploy AI-powered web applications

Phase 1: Python Foundation

Why This Phase?

Django, DRF, आणि FastAPI सगळ्यांसयठी strong Python base mandatory आहे. OOP आणि async/await नयही केलां तर framework code समजियर नयही.

1.1 Core Python Concepts

- Variables, data types, operators, f-strings
- Conditionals, loops (for/while), comprehensions
- Functions — args, kwargs, default params, *args/**kwargs
- Scope — local, global, nonlocal
- Exception handling — try/except/finally, custom exceptions
- File I/O — open(), read/write, context managers (with)

1.2 Object-Oriented Programming (OOP)

- Classes and objects — __init__, self, attributes
- Inheritance — single, multiple, super()
- Dunder/magic methods — __str__, __repr__, __len__, __eq__
- Class methods (@classmethod), static methods (@staticmethod)
- Properties — @property, getter/setter
- Abstract classes (ABC module)
- Dataclasses — @dataclass decorator

1.3 Advanced Python

- Decorators — function decorators, chaining decorators
- Generators — yield, generator expressions
- Iterators — `__iter__`, `__next__`
- Context managers — `__enter__`, `__exit__`, `contextlib`
- Type hints — `str`, `int`, `list[dict]`, `Optional`, `Union`, `Any`
- Lambda functions, `map()`, `filter()`, `zip()`
- Closures and first-class functions

1.4 Async Python

- Synchronous vs asynchronous programming concept
- `async def` and `await` keyword
- `asyncio` basics — event loop, coroutines
- `async with`, `async for`
- `asyncio.gather()` — running tasks concurrently

1.5 Modules & Packaging

- Virtual environments — `python -m venv`, activation
- `pip` — `install`, `freeze`, `requirements.txt`
- Modules vs packages — `__init__.py`
- Common stdlib — `os`, `sys`, `pathlib`, `json`, `datetime`, `re`, `collections`
- Third-party packages — `requests`, `Pillow`, `python-dotenv`

1.6 Testing Basics

- `unittest` — `TestCase`, `setUp`, `tearDown`
- `pytest` — test discovery, fixtures, assertions
- Writing simple unit tests for functions

Phase 1 — Mini Projects

- **CLI Calculator:** OOP-based, handles errors gracefully
- **File Organizer:** Reads directory, sorts files by extension
- **JSON Data Processor:** Read/filter/transform JSON data

Phase 1 — Resources

- **Official Python Docs:** docs.python.org/3/tutorial
- **Real Python:** realpython.com — beginner to intermediate articles
- **Automate the Boring Stuff:** automatetheboringstuff.com — free book

Phase 2: Django Core

Why Django First?

Django ણિકલ્યવર ORM, auth, forms સગळाંકય package મધ્ેનમळतां. हे concepts FastAPI सयठी पि लयगतयत — फक्त implementation different असतो.

2.1 Django Setup & Project Structure

- Installation — `pip install django, django-admin startproject`
- Project structure — `settings.py, urls.py, wsgi.py, asgi.py`
- App creation — `python manage.py startapp`
- Development server — `python manage.py runserver`
- `settings.py` — `INSTALLED_APPS, DATABASES, STATIC_URL, MEDIA_ROOT` • Environment-specific settings (dev vs production)

2.2 MVT Pattern (Model-View-Template)

- MVT concept — how it differs from MVC
- Django request-response lifecycle
- URL dispatcher — `urls.py, path(), include()`
- URL namespacing — `app_name, reverse()`
- URL parameters — path converters (int, str, slug, uuid)

2.3 Models & Django ORM

- Model fields — `CharField, IntegerField, DateTimeField, ForeignKey, ManyToManyField` • Field options — `null, blank, default, unique, choices`
- Model Meta — `ordering, verbose_name, db_table`
- Migrations — `makemigrations, migrate, showmigrations`
- Squashing migrations
- CRUD operations — `.create(), .save(), .all(), .filter(), .get(), .delete()`
- QuerySet methods — `.order_by(), .values(), .distinct(), .count(), .exists()` • Lookups — `_contains, _startswith, _gte, _lte, _in, _isnull`
- Q objects — complex OR/AND queries
- F expressions — database-level field operations
- `select_related()` — ForeignKey optimization
- `prefetch_related()` — ManyToMany optimization
- `annotate()` and `aggregate()` — `Sum, Count, Avg, Max, Min`
- Custom model managers
- Signals — `pre_save, post_save, pre_delete`

2.4 Views

- Function-based views (FBV) — request, response
- Class-based views (CBV) — `View, TemplateView, ListView, DetailView, CreateView,`

UpdateView, DeleteView

- Mixins — LoginRequiredMixin, PermissionRequiredMixin
- HttpResponse, JsonResponse, redirect(), render()
- get_object_or_404(), get_list_or_404()
- Request object — GET, POST, FILES, user, method, headers
- Decorators — @login_required, @permission_required

2.5 Templates

- Django template language (DTL)
- Template inheritance — {% extends %}, {% block %}
- Template tags — {% if %}, {% for %}, {% url %}, {% static %}, {% include %} •
- Template filters — |date, |truncatechars, |default, |length
- Custom template tags and filters
- Context processors

2.6 Forms

- Django Form class — fields, widgets, validation
- ModelForm — Meta class, fields, exclude
- Form validation — clean(), clean_<fieldname>()
- Formsets — multiple forms on one page
- File upload handling
- CSRF protection

2.7 Authentication & Authorization

- User model — AbstractUser, AbstractBaseUser
- Custom User model (do this first in any project)
- Login, logout, signup views
- Password hashing and reset flow
- Permissions — add, change, delete, view
- Groups and permission assignment
- Session management
- allauth library for social auth (Google, GitHub)

2.8 Admin Panel

- Registering models — admin.site.register()
- ModelAdmin — list_display, list_filter, search_fields
- Inline admin — TabularInline, StackedInline
- Custom admin actions
- Admin site customization

2.9 Static & Media Files

- STATIC_URL, STATICFILES_DIRS, STATIC_ROOT

- MEDIA_URL, MEDIA_ROOT — user-uploaded files
- collectstatic command
- Serving static files in development vs production
- ImageField, FileField — storing & serving user uploads

2.10 Middleware

- Middleware concept and execution order
 - Built-in middleware — SecurityMiddleware, SessionMiddleware, CsrfViewMiddleware •
- Writing custom middleware

Phase 2 — Projects

- **Blog App:** Full CRUD, categories, tags, user auth, rich text (CKEditor/Quill) •
- Task Manager:** User login, tasks with priorities, deadlines, status
- **E-commerce Store (basic):** Products, categories, cart logic (no payment yet)

Phase 2 — Resources

- **Official Django Docs:** docs.djangoproject.com — read everything in "The Model Layer" and "The View Layer"
 - **William Vincent:** "Django for Beginners" book —.djangoforbeginners.com •
- TestDriven.io:** Advanced Django guides with Docker and testing

Phase 3: Django REST Framework

(DRF)

Why DRF?

DRF = Django चा API बनवण्याचा tool. React, Flutter, mobile apps सयठी API लयगतां. हेणिकल्यवर तू proper backend developer होतोस.

3.1 DRF Basics

- DRF installation and setup — REST_FRAMEWORK settings
- API views — APIView class
- Response object — Response(data, status=)
- Status codes — from rest_framework import status
- Request object in DRF — request.data, request.user
- Format suffixes — .json, .api browsable API

3.2 Serializers

- Serializer class — fields, validate(), to_representation()

- `ModelSerializer` — `Meta`, `fields`, `read_only_fields`, `extra_kwargs`
- Nested serializers — `ForeignKey` relationships
- `SerializerMethodField` — custom computed fields
- `create()` and `update()` overrides
- Validation — `validate_<field>()`, `validate()` for cross-field
- `HyperlinkedModelSerializer` — `HATEOAS` APIs

3.3 Views & ViewSets

- Function-based API views — `@api_view` decorator
- Generic views — `ListAPIView`, `CreateAPIView`, `RetrieveAPIView`, `UpdateAPIView`, `DestroyAPIView`
- Concrete generic views — `ListCreateAPIView`, `RetrieveUpdateDestroyAPIView` •
- `ViewSets` — `ModelViewSet`, `ReadOnlyModelViewSet`
- Routers — `DefaultRouter`, `SimpleRouter`
- Custom actions — `@action` decorator

3.4 Authentication in DRF

- `SessionAuthentication` — browser-based
- `BasicAuthentication` — development only
- `TokenAuthentication` — DRF built-in
- `JWT Authentication` — `django-rest-framework-simplejwt`
- `JWT flow` — obtain token, refresh token, verify token
- `OAuth2` — `django-oauth-toolkit` (advanced)

3.5 Permissions

- Built-in — `IsAuthenticated`, `IsAdminUser`, `AllowAny`, `IsAuthenticatedOrReadOnly` •
- Object-level permissions — `has_object_permission()`
- Custom permission classes
- `DEFAULT_PERMISSION_CLASSES` setting

3.6 Filtering, Searching, Pagination

- `django-filter` — `FilterSet`, field-level filtering
- DRF `SearchFilter` — `search_fields`
- `OrderingFilter` — `ordering_fields`
- `Pagination` — `PageNumberPagination`, `LimitOffsetPagination`, `CursorPagination`
- Custom pagination class

3.7 Throttling & Versioning

- `Throttling` — `AnonRateThrottle`, `UserRateThrottle`, `ScopedRateThrottle` •
- Custom throttle classes
- API versioning strategies — URL path, query param, header
- `NamespaceVersioning`, `URLPathVersioning`

3.8 Testing DRF APIs

- APIClient — get(), post(), put(), patch(), delete()
- force_authenticate() — bypass auth in tests
- Testing serializers, views, permissions
- Factory Boy — test data generation

Phase 3 — Projects

- **Blog REST API:** Full CRUD, JWT auth, pagination, filtering, Swagger docs
- **Social Media API:** Users, posts, likes, follow/unfollow, feed endpoint
- **Library API:** Books, authors, borrow history, user management

Phase 4: Database & Advanced ORM

4.1 PostgreSQL

- Installation, psql CLI basics
- Connecting Django to PostgreSQL — psycopg2
- Database design — normalization (1NF, 2NF, 3NF)
- Data types in PostgreSQL — JSON, JSONB, ARRAY, UUID
- PostgreSQL-specific Django fields — JSONField, ArrayField, HStoreField

4.2 Advanced ORM

- Complex Q object queries — nested, negation (~Q)
- Database functions — Coalesce, Cast, ExtractYear, TruncMonth
- Window functions — Rank, RowNumber, Lead, Lag
- Raw SQL — raw(), connection.cursor()
- Database transactions — atomic(), savepoints
- Bulk operations — bulk_create(), bulk_update()
- Custom SQL indexes on models
- Model inheritance — abstract, multi-table, proxy models

4.3 Performance Optimization

- Django Debug Toolbar — N+1 query detection
- Query optimization — avoiding N+1 with select_related/prefetch_related
- Database indexing — db_index=True, Index class
- only() and defer() — limiting fetched fields
- Caching strategies — per-view, per-object, low-level
- django-cachalot — ORM-level caching

4.4 Redis & Caching

- Redis setup — docker or local install
- django-redis — CACHES setting
- `cache.set()`, `cache.get()`, `cache.delete()`
- `cache_page()` decorator — full-page caching
- Caching querysets
- Session storage in Redis

4.5 Celery — Async Task Queue

- Celery concept — task queue, worker, broker
- Setup — `celery.py`, `CELERY_BROKER_URL` (Redis)
- `@shared_task` decorator
- Running celery worker
- celery beat — periodic tasks (cron-like) •

Task chaining and groups

- Monitoring — Flower dashboard

Phase 5: Frontend + Deployment

(Fullstack)

5.1 React Basics (for Django API consumers)

- React setup — create-react-app or Vite
- Components — functional components
- JSX syntax
- `useState`, `useEffect` hooks
- Props and component communication
- React Router — client-side routing
- Axios — API calls, interceptors, error handling
- JWT storage — `localStorage` / `httpOnly` cookies (security trade-offs)
- Protected routes in React

5.2 CORS & Security

- CORS concept — same-origin policy
- `django-cors-headers` — `CORS_ALLOWED_ORIGINS`
- Content Security Policy basics
- HTTPS in production
- `django-environ` — environment variable management

5.3 Docker

- Docker concepts — image, container, volume, network

- Dockerfile — FROM, RUN, COPY, CMD, EXPOSE
- docker-compose.yml — Django + PostgreSQL + Redis services
- .dockerignore file
- Running Django in Docker
- Environment variables in Docker — .env files

5.4 Deployment

- Deployment options — Railway, Render, Heroku, DigitalOcean
- Gunicorn — WSGI server for Django
- Nginx — reverse proxy, static file serving
- Static files in production — WhiteNoise or S3
- collectstatic in deployment
- PostgreSQL in production — managed DB services
- Environment variables in production
- ALLOWED_HOSTS, DEBUG=False
- SSL/HTTPS setup

5.5 CI/CD Basics

- GitHub Actions — workflow YAML
- Automated testing on push
- Auto-deploy on merge to main
- Environment secrets in GitHub

5.6 Testing (Comprehensive)

- pytest-django — configuration, fixtures
- Model tests, view tests, API tests
- Mock — mocking external APIs
- Coverage — measuring test coverage
- Selenium / Playwright — end-to-end testing (optional)

Phase 5 — Capstone Project

Build ONE complete project: Django REST API backend + React frontend + Docker + Deployed on Render/Railway. This should be on your GitHub and resume.

- **Option A:** Full Blog Platform — posts, comments, likes, user profiles, image upload, JWT auth
- **Option B:** Job Board — companies, listings, applications, status tracking, email notifications
- **Option C:** Task Management SaaS — workspaces, teams, tasks, Kanban board, due dates

Phase 6: FastAPI Transition

Django नंतर FastAPI Transition

Django/DRF शिकल्यावर FastAPI खूप easy वाटतो. Pydantic = DRF Serializers, Path operations = URL views, Dependency injection = Mixins.

6.1 FastAPI Basics

- FastAPI installation — `pip install fastapi uvicorn`
- Running server — `uvicorn main:app --reload`
- Path operations — `@app.get()`, `@app.post()`, `@app.put()`, `@app.delete()` • Path parameters and query parameters
- Request body — `BaseModel`
- Response model — `response_model=`
- Status codes — `status_code=201`
- Auto docs — `/docs` (Swagger), `/redoc`

6.2 Pydantic Models

- `BaseModel` — fields with type hints
- `Field()` — `default`, `description`, `gt`, `lt`, `min_length`
- Validators — `@validator`, `@root_validator`
- Nested models
- Optional fields — `Optional[str] = None`
- Response models — excluding fields with `response_model_exclude`

6.3 Database with SQLAlchemy / SQLAlchem

- `SQLModel` — combines `SQLAlchemy` + `Pydantic` (recommended with FastAPI)
- Creating tables, sessions
- CRUD operations async style
- Alembic migrations with FastAPI
- Dependency injection for DB session — `Depends(get_db)`

6.4 Auth in FastAPI

- `OAuth2PasswordBearer`
- JWT — `python-jose` library
- Password hashing — `passlib`
- Login endpoint — returning access token
- Protected routes — `Depends(get_current_user)`

6.5 Advanced FastAPI

- Dependency injection — `Depends()`
- Background tasks — `BackgroundTasks`

- Middleware — BaseHTTPMiddleware
- WebSockets
- File uploads — UploadFile, File
- CORS — CORSMiddleware
- Exception handlers — HTTPException, custom handlers
- Routers — APIRouter for modular structure • Lifespan events — startup/shutdown

Phase 6 — Projects

- **URL Shortener API:** FastAPI + Redis + statistics tracking •
- Notes API:** Full CRUD, JWT auth, search, SQLAlchemy •
- Real-time Chat:** WebSockets, rooms, message history

Phase 7: AI / ML Integration (Main

Goal)

तुझ Main Target

हा phase "शिकून संपवायचा" नाही — हा ongoing आहे. Django/FastAPI foundation वर AI features build करत जायचं.

7.1 Python for Data & ML

- NumPy — arrays, vectorized operations, broadcasting
- Pandas — DataFrames, Series, groupby, merge, pivot
- Matplotlib & Seaborn — data visualization
- Scikit-learn — preprocessing, train/test split, pipelines
- Jupyter Notebooks — exploratory analysis

7.2 Machine Learning Fundamentals

- Supervised vs unsupervised learning
- Regression — Linear, Ridge, Lasso
- Classification — Logistic Regression, SVM, Random Forest, XGBoost
- Clustering — K-Means, DBSCAN
- Model evaluation — accuracy, precision, recall, F1, ROC-AUC
- Cross-validation, hyperparameter tuning (GridSearchCV)
- Feature engineering and selection
- Saving models — joblib, pickle

7.3 Serving ML Models via FastAPI

- Load trained model at startup (lifespan)
- Prediction endpoint — POST /predict
- Input validation with Pydantic
- Batch prediction endpoints
- Model versioning strategy
- Performance — async inference, model caching

7.4 Large Language Models (LLMs) & APIs

- OpenAI API — chat completions, embeddings, vision
- Anthropic Claude API — messages API
- Prompt engineering — system prompts, few-shot examples
- Streaming responses — Server-Sent Events (SSE)
- Function calling / tool use
- Token limits and cost management
- Rate limiting and retry logic

7.5 LangChain

- LangChain concepts — chains, agents, tools
- LLM wrappers — ChatOpenAI, ChatAnthropic
- Prompt templates — ChatPromptTemplate, PromptTemplate
- Chains — LLMChain, SequentialChain
- Memory — ConversationBufferMemory, ConversationSummaryMemory •
- Agents — ReAct, OpenAI Functions agent
- Tools — custom tools, web search, calculator
- Callbacks and streaming

7.6 RAG (Retrieval-Augmented Generation)

- RAG concept — why LLMs need external knowledge
- Text splitting — RecursiveCharacterTextSplitter
- Embeddings — OpenAI, HuggingFace sentence-transformers
- Vector databases — ChromaDB (local), Pinecone (cloud), Weaviate
- Similarity search — cosine similarity, FAISS
- RAG pipeline — load → split → embed → store → retrieve → generate
- PDF, web page, database as knowledge sources

7.7 Hugging Face & Open-Source Models

- Hugging Face Hub — finding models
- Transformers library — pipeline() API
- Text classification, NER, sentiment analysis pipelines
- Text generation — GPT-2, LLaMA, Mistral
- Image models — CLIP, Stable Diffusion (basics)
- Running models locally vs API

- Gradio — quick UI for ML models

7.8 AI-Powered Django/FastAPI Apps

- AI chatbot with conversation history (Django + LangChain)
- Document Q&A system (RAG + FastAPI)
- AI image analysis endpoint (OpenAI Vision)
- Async AI tasks with Celery (long-running inference)
- Streaming AI responses to frontend (SSE)
- AI content moderation in Django

Phase 7 — Capstone AI Projects

- **AI Chatbot App:** Django + LangChain + React. Conversation memory, streaming, multiple personas.
- **Document QA System:** Upload PDF → RAG pipeline → Ask questions. FastAPI + ChromaDB + OpenAI.
- **Resume Screener:** Upload resumes, JD input, AI ranking. Celery for batch processing.
- **Image Caption API:** Upload image → OpenAI Vision → return description + tags. FastAPI.
- **Code Review Bot:** GitHub webhook → AI reviews PR → posts comment. Django + LangChain.

Bonus: Production Skills & Career Prep

Production-Grade Code

- Logging — Python logging, structured logs, Sentry error tracking
- Monitoring — health check endpoints, uptime monitoring
- API rate limiting — production throttling strategies
- Security — OWASP Top 10 awareness, SQL injection prevention, XSS, CSRF
- Load testing — locust, k6
- Horizontal scaling concepts — load balancer, stateless design

Tools Every Python Dev Should Know

- Git — branching strategy (GitFlow), rebase vs merge, PR workflow
- Pre-commit hooks — black (formatter), isort, flake8, mypy
- Makefile — automating common dev commands
- Postman / Thunder Client — API testing
- DBeaver / TablePlus — database GUI
- Docker Desktop

Resume & Portfolio Tips

- Minimum 3 GitHub projects — one Django, one FastAPI, one AI-powered
- Each project needs: README with screenshots, live demo link, clear description

Contribute to open-source — even docs fixes count

- Write tech blog posts — explains concepts to juniors
- LinkedIn — Python Developer, Django, FastAPI, AI/ML keywords •

LeetCode — 50-100 medium problems (for product company interviews)

Quick Reference — Complete Syllabus Summary

Python Foundation 3–4 weeks OOP, async, type hints, modules, testing **Django Core** 4–5 weeks

MVT, ORM, views, forms, auth, admin, static **Django REST Framework** 3–4 weeks Serializers,

ViewSets, JWT, permissions, filtering

Database & Advanced ORM

Celery

2–3 weeks PostgreSQL, query optimization, Redis,

Frontend + Deployment 3–4 weeks React basics, Docker, Render deploy, CI/CD, testing **FastAPI**

Transition 2–3 weeks Path ops, Pydantic, SQLAlchemy, async, WebSockets

AI / ML Integration Ongoing NumPy/Pandas, ML models, LangChain, RAG, LLMs, HuggingFace

Total estimated time: 18–24 months self-paced | 10–12 months with 4–6 hrs/day commitment

Focus on building projects at every phase — theory alone will not make you job-ready.